

Da Compreensão Conceitual dos Fluxos de Processo (Linear, Iterativo, Evolucionário e Paralelo) à Aplicação Prática dos Modelos Prescritivos (Cascata, Prototipação e Espiral)

PROCESSO DE SOFTWARE — Professor: Me. Sergio Souza Novak —
Faculdade de Engenharia de Software



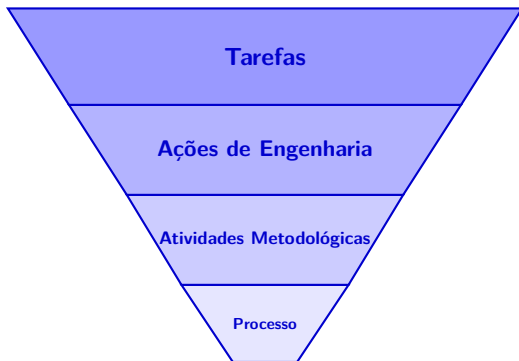
1 O Processo de Software

2 Fluxos de Processo

- Fluxo Linear
- Fluxo Iterativo
- Fluxo Evolucionário
- Fluxo Paralelo

3 Modelos de Processo Prescritivo

- Modelo Cascata
- Modelo de Prototipação
- Modelo Espiral (Evolucionário)



- **Processo**: Define a estrutura global.
- **Atividades**: Fases metodológicas amplas (ex: Comunicação, Modelagem).
- **Ações**: Passos dentro de uma atividade (ex: Projeto arquitetural).
- **Tarefas**: Trabalho diário específico da equipe (ex: Criar diagrama).

1. Processo de Software

Exemplo: *Desenvolvimento com Processo Ágil (ex: Scrum)*

2. Atividade Metodológica

Exemplo: *Comunicação (Levantamento de Requisitos)*

3. Ação de Engenharia

Exemplo: *Elicitação e Análise de Requisitos*

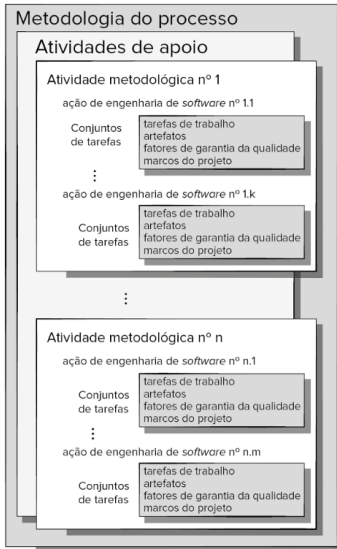
4. Tarefas (O Trabalho Diário)

- Agendar reunião com o diretor.
- Conduzir entrevista sobre necessidades.
- Escrever as *User Stories*.
- Validar as anotações com a equipe.

Atividades de apoio de um processo

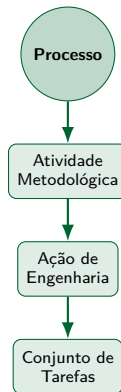


Processo de *software*



O processo de software é uma estrutura que organiza as atividades de desenvolvimento:

- **Atividade Metodológica:** Ações de trabalho que determinam o relacionamento entre si e com o processo global.
- **Ações de Engenharia:** Conjuntos de tarefas focadas em objetivos específicos do projeto.
- **Conjunto de Tarefas:** Define o trabalho real:
 - Tarefas de trabalho a serem completadas.
 - Artefatos de software produzidos.
 - Fatores de garantia da qualidade exigidos.
 - Marcos (milestones) para indicar progresso.



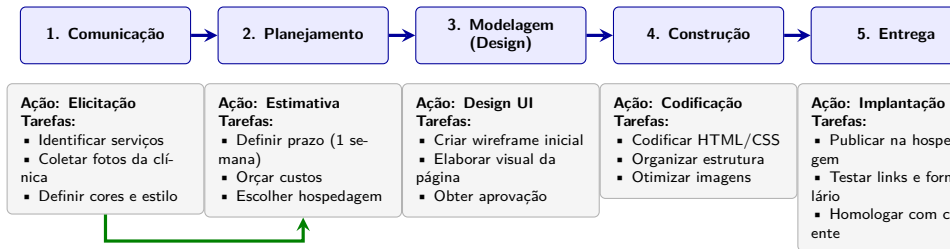
Um **fluxo de processo linear** executa cada uma das cinco atividades metodológicas em sequência direta, iniciando na comunicação e concluindo na entrega.



- **Sequência Rígida:** Nenhuma etapa é iniciada até que a anterior esteja concluída.
- **Adequabilidade:** Ideal para projetos simples, bem compreendidos e com requisitos altamente estáveis que não sofrerão alterações durante o desenvolvimento.

Exemplo Prático de Fluxo Linear

Cenário: Desenvolvimento de um *Site Institucional Estático* para uma clínica médica.



Fluxo linear: Comunicação → Planejamento → Modelagem → Construção → Entrega

Estrutura da Atividade Metodológica: Construção

Exemplo de instanciação da hierarquia do processo para a atividade de **Construção**:

Atividades de Apoio (Gerenciamento, Qualidade)

Atividade Metodológica: Construção

Ação de Eng. de Software: Codificação

Conjunto de Tarefas (Codificação):

- **Tarefas de trabalho:** Codificar a estrutura (HTML) e estilo (CSS)
- **Artefatos produzidos:** Arquivos .html, .css e scripts
- **Garantia da qualidade:** Validação de padrões W3C e linting
- **Marcos de progresso:** Protótipo visual responsivo concluído

Ação de Eng. de Software: Testes

Conjunto de Tarefas (Testes):

- **Tarefas de trabalho:** Executar testes de navegabilidade e links
- **Artefatos produzidos:** Relatório de testes e registro de erros
- **Garantia da qualidade:** Cobertura de testes em múltiplos navegadores
- **Marcos de progresso:** Código homologado e pronto para entrega

Estrutura da Atividade Metodológica: Comunicação

Exemplo de instanciação da hierarquia do processo para a atividade de **Comunicação**:

Atividades de Apoio (Gerenciamento, Qualidade)

Atividade Metodológica: Comunicação

Ação de Eng. de Software: Concepção (Elicitação)

Conjunto de Tarefas (Concepção):

- **Tarefas de trabalho:** Realizar reuniões e coletar requisitos iniciais
- **Artefatos produzidos:** Atas de reunião, lista preliminar de requisitos
- **Garantia da qualidade:** Validação de escopo com os principais stakeholders
- **Marcos de progresso:** Necessidades e escopo inicial documentados

Ação de Eng. de Software: Negociação de Escopo

Conjunto de Tarefas (Negociação):

- **Tarefas de trabalho:** Alinhar expectativas, prazos e custos do projeto
- **Artefatos produzidos:** Proposta técnica, termo de especificações acordadas
- **Garantia da qualidade:** Análise de viabilidade técnica e financeira
- **Marcos de progresso:** Contrato assinado ou escopo final formalizado

Estrutura da Atividade Metodológica: Entrega

Exemplo de instanciação da hierarquia do processo para a atividade de **Entrega** (Implantação):

Atividades de Apoio (Gerenciamento, Qualidade)

Atividade Metodológica: Entrega

Ação de Eng. de Software: Implantação (Deployment)

Conjunto de Tarefas (Implantação):

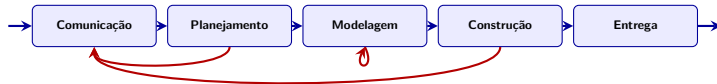
- **Tarefas de trabalho:** Publicar arquivos no servidor e configurar DNS
- **Artefatos produzidos:** Site no ar em ambiente de produção
- **Garantia da qualidade:** Testes de fumaça pós-deploy e validação de SSL
- **Marcos de progresso:** Site público e acessível de forma estável

Ação de Eng. de Software: Feedback e Suporte

Conjunto de Tarefas (Feedback):

- **Tarefas de trabalho:** Apresentar ao cliente e coletar avaliações/ajustes
- **Artefatos produzidos:** Termo de recebimento, lista de melhorias futuras
- **Garantia da qualidade:** Avaliação da satisfação e checklist de aceitação
- **Marcos de progresso:** Homologação final e encerramento do projeto

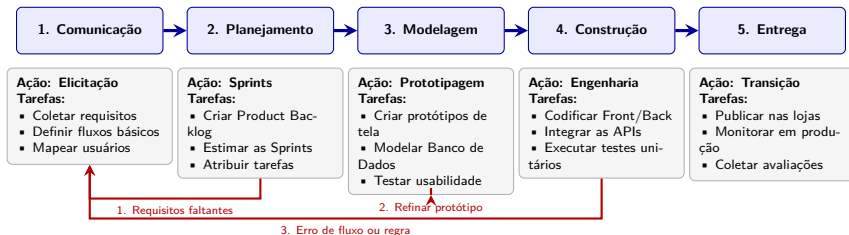
Um **fluxo de processo iterativo** repete uma ou mais atividades metodológicas (sendo executadas em ciclos) antes de prosseguir para a próxima, permitindo o refinamento contínuo do software.



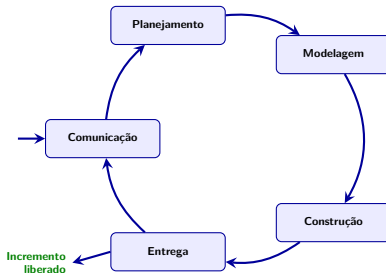
- **Caminhos de Retorno:** As setas vermelhas indicam loops de feedback onde o fluxo retorna a etapas anteriores para correções ou novos detalhes.
- **Adequabilidade:** Ideal para projetos complexos onde os requisitos não são totalmente claros no início.

Exemplo Prático de Fluxo Iterativo

Cenário: Desenvolvimento de um *Aplicativo de Entrega de Comida (Delivery)*.



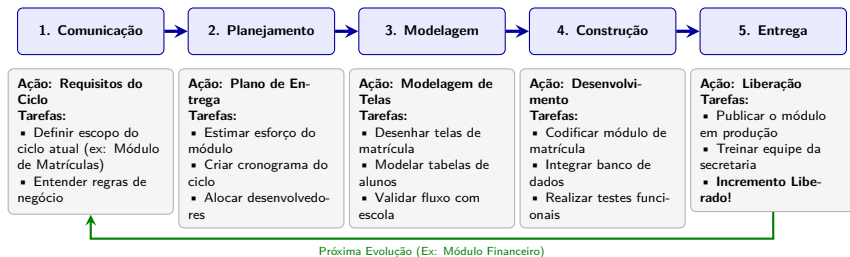
O **fluxo de processo evolucionário** é circular. Ele é caracterizado pela entrega de versões cada vez mais completas do software (incrementos) a cada ciclo completo executado.



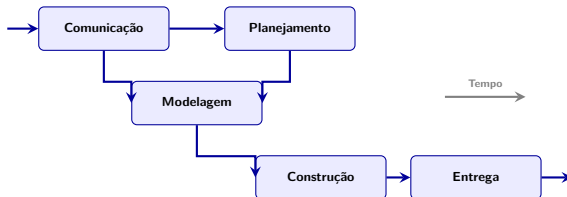
- **Ciclo de Feedback Contínuo:** O software *evolui* à medida que o cliente utiliza cada incremento e sugere melhorias para a próxima iteração.
- **Adequabilidade:** Projetos de grande porte, sistemas SaaS, startups e produtos em constante mutação.

Exemplo Prático de Fluxo Evolucionário

Cenário: Desenvolvimento de um *Sistema de Gestão Escolar* (SaaS).



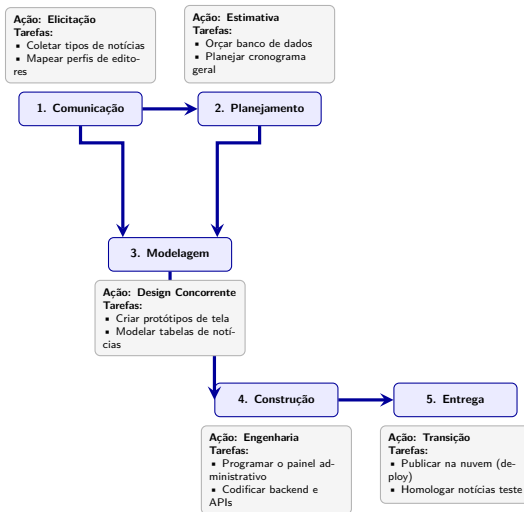
O **fluxo de processo paralelo** ocorre quando uma ou mais atividades são executadas em concorrência (paralelo) com outras que já foram iniciadas no projeto.



- **Concorrência:** Representa a realidade de equipes de desenvolvimento onde design de UI (Modelagem) pode iniciar antes de todo o Planejamento estar 100% fechado.
- **Sincronização:** Exige controle rígido (atividades de apoio) para garantir consistência no momento da integração (Construção).

Exemplo Prático de Fluxo Paralelo

Cenário: Desenvolvimento de um *Portal de Notícias* dinâmico.



Uma equipe de software precisa definir de forma adequada quais ações compõem cada atividade metodológica antes de iniciar o trabalho.

Questão-Chave: Quais ações são apropriadas para uma atividade metodológica?

Essa escolha deve ser feita sob medida, considerando:

- **A natureza do problema** a ser solucionado.
- **As características das pessoas** que farão o trabalho.
- **Os envolvidos (stakeholders)** no projeto de software.

Diretriz: A complexidade do projeto determina a densidade e o número de ações e tarefas do processo.

A atividade de **Comunicação** pode variar drasticamente dependendo do escopo do projeto:

Projeto Simples (Um Cliente)

Uma conversa telefônica rápida (Ação Única) envolvendo as seguintes tarefas de trabalho:

- 1 Contatar o envolvido via telefone.
- 2 Discutir requisitos e gerar anotações.
- 3 Organizar notas em relação escrita de requisitos.
- 4 Enviar e-mail para revisão e aprovação.

Projeto Complexo (Múltiplos Clientes)

Requisitos concorrentes ou conflitantes que exigem seis ações de engenharia distintas:

- **Concepção:** Iniciar entendimento.
- **Levantamento:** Elicitar necessidades.
- **Elaboração:** Refinar requisitos.
- **Negociação:** Ajustar conflitos.
- **Especificação:** Documentar.
- **Validação:** Homologar.

Modelos de processo prescritivos definem um conjunto prescrito de elementos de processo e um fluxo de trabalho previsível.

- **Foco Central:** Estruturar e ordenar o desenvolvimento de software.
- **Fluxo de Trabalho:** Atividades e tarefas ocorrem de forma sequencial com diretrizes claras para mensurar o progresso.

Questões de Reflexão para a Engenharia de Software:

- *Esses modelos são adequados para um mundo de software dinâmico que se alimenta de mudanças frequentes?*
- *Se rejeitarmos esses modelos tradicionais (e sua ordem implícita) por algo menos estruturado, tornaremos impossível atingir a coordenação e coerência no trabalho?*

Há casos em que os requisitos de um problema são **bem compreendidos** e o trabalho flui da comunicação à entrega de modo relativamente **linear**.

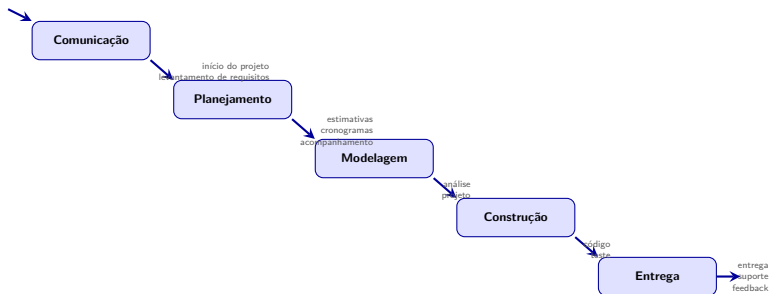
Essa situação ocorre quando:

- Adaptações ou aperfeiçoamentos **bem definidos** precisam ser feitos em um sistema existente.
Ex.: adaptação de software contábil devido a mudanças em normas governamentais.
- Novos esforços de desenvolvimento com requisitos **bem definidos e razoavelmente estáveis**.

O **modelo cascata** (também chamado de *modelo sequencial linear*) sugere uma abordagem **sequencial e sistemática** para o desenvolvimento de software:

Especificação de requisitos → Planejamento → Modelagem → Construção → Entrega → Suporte contínuo

Modelo Cascata – Diagrama (Figura 2.3)



O fluxo do modelo cascata é **estritamente sequencial**: cada fase deve ser completada antes da próxima iniciar.

O modelo cascata é o **paradigma mais antigo** da engenharia de software. Ao longo das últimas cinco décadas, as críticas fizeram até seus mais árdios defensores questionarem sua eficácia.

Principais Críticas:

- Projetos reais **raramente** seguem um fluxo sequencial puro.
- É difícil para o cliente estabelecer **todos os requisitos explicitamente** no início do projeto.
- O cliente deve ter **paciência**: uma versão funcional do software só estará disponível em uma etapa avançada do cronograma.
- Um erro grave, se não detectado cedo, pode ser **desastroso** nas fases finais.

Quando usar? Apenas quando os requisitos são **fixos e bem compreendidos** desde o início, e quando o trabalho deve ser completado de modo **linear até a conclusão**.

Problema 1: Fluxo Sequencial Irreal



Projetos reais raramente seguem o fluxo sequencial proposto pelo modelo cascata.

- Na prática, as atividades se sobrepõem e interagem entre si.
- Mudanças no meio do caminho exigem **retornos a fases anteriores**, algo que o modelo não prevê naturalmente.
- A linearidade estrita é uma **idealização** que não reflete a complexidade de projetos reais.

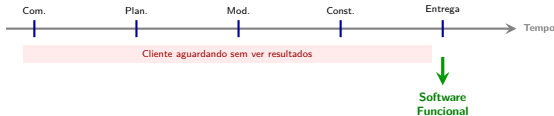
Problema 2: Requisitos Iniciais Incompletos



Com frequência, é difícil para o cliente estabelecer explicitamente todas as necessidades no início da maioria dos projetos.

- O cliente pode não ter clareza total sobre o que deseja.
- Requisitos são **descobertos progressivamente**, à medida que o projeto evolui e protótipos são apresentados.
- O modelo cascata assume que os requisitos são **fixos e completos** desde o começo – premissa frequentemente falsa.

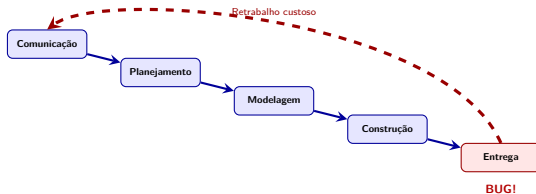
Problema 3: Versão Operacional Tardia



O cliente deve ter paciência: uma versão operacional do programa **não estará disponível** antes de se estar próximo ao final do projeto.

- O cliente não consegue validar o software durante o desenvolvimento.
- Se o resultado final não corresponder às expectativas, o **retrabalho será extenso e custoso**.
- Modelos iterativos e incrementais resolvem isso com entregas parciais ao longo do projeto.

Problema 4: Erros Detectados Tardiamente



Erros graves podem não ser detectados até o programa operacional ser revisto pelo cliente na fase final.

- Um erro de requisito inserido na fase de Comunicação só aparece na Entrega.
- O custo de correção cresce **exponencialmente** quanto mais tarde o erro é descoberto.
- Revisões intermediárias e testes contínuos minimizam esse risco.

Conclusão: Modelo Cascata e o Software Moderno

Hoje, o trabalho com software tem um **ritmo acelerado** e está sujeito a uma cadeia de **mudanças intermináveis** em características, funções e conteúdo de informações.

O modelo cascata é frequentemente inadequado para esse tipo de trabalho.

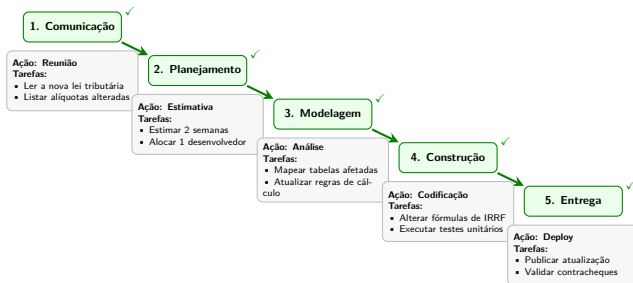
Alternativas modernas que lidam melhor com mudanças:

- **Modelos Iterativos:** Permitem retornos planejados entre fases.
- **Modelos Evolucionários:** Entregam incrementos funcionais a cada ciclo.
- **Modelos Ágeis:** Abraçam a mudança como parte natural do processo.

Lição: O modelo cascata ainda pode ser útil em contextos específicos, mas **não deve ser a escolha padrão** para a maioria dos projetos modernos de software.

Exemplo: Cascata Adequado

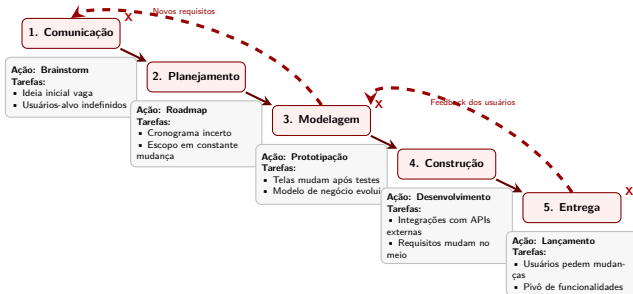
Cenário: Atualização do módulo de *Folha de Pagamento* para atender nova legislação tributária federal.



Por que funciona? Requisitos **fixos** (definidos pela legislação), escopo **pequeno e claro**, sem ambiguidade – fluxo linear é natural.

Exemplo: Cascata Inadequado

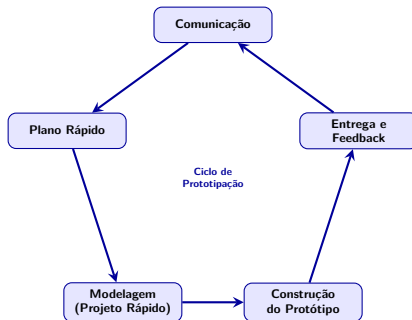
Cenário: Desenvolvimento de um *Aplicativo de Caronas Compartilhadas* (startup inovadora).



Por que falha? Requisitos **vagos e mutáveis**, feedback constante dos usuários, necessidade de **pivôs rápidos** – o fluxo linear trava o projeto.

Modelo de Processo de Prototipação

O paradigma da **prototipação** é indicado quando o cliente define objetivos gerais, mas **não detalha requisitos** para funções e recursos; ou quando o desenvolvedor está **inseguro** quanto a algoritmos, adaptabilidade ou interação com o usuário.



- O protótipo serve como mecanismo para **identificar requisitos**.
- O ciclo se repete até que o cliente e o desenvolvedor concordem sobre o que é necessário.

Prototipação: Aplicação e Características

A prototipação pode ser utilizada como **modelo isolado** (*stand-alone*) ou como **técnica complementar** dentro de qualquer outro modelo de processo.

Quando usar a prototipação?

- Requisitos estão **obscuros ou incompletos**.
- O cliente não sabe exatamente o que quer.
- O desenvolvedor precisa validar **viabilidade técnica** (algoritmo, interface, desempenho).

Benefícios:

- Auxilia os envolvidos a **compreender melhor** o que será construído.
- Reduz o risco de construir o software errado.
- Permite **feedback visual e tangível** desde cedo.

Atenção: O protótipo é descartável ou evolucionário. Em ambos os casos, deve-se evitar que um protótipo mal construído se torne o produto final sem a devida engenharia.

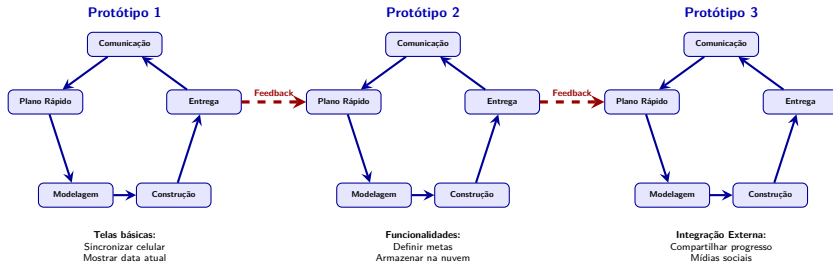


Cenário: Desenvolvimento de um aplicativo de *fitness* para dispositivo móvel usando **prototipação incremental**.

O aplicativo não é construído de uma só vez. Em vez disso, funções são adicionadas em **ciclos progressivos**, refinando a interface e o funcionamento com base no **feedback dos clientes**.

- Permite que o cliente valide a interface e usabilidade **antes** do desenvolvimento completo das funções.
- **Reduz riscos:** se uma funcionalidade não atender às expectativas, ela é ajustada rapidamente.
- Ao final de múltiplos protótipos, o sistema alcança a maturidade técnica e a aprovação do usuário.

Ciclos de Prototipação: App Fitness



Cada novo protótipo modifica e melhora as telas da interface com base no **feedback dos usuários**, completando um novo ciclo metodológico a cada iteração.

Apesar de excelente para lidar com incertezas, a prototipação pode gerar problemas se não for bem gerenciada:

1. Falsa ilusão do cliente

- O cliente vê o protótipo funcionando e não entende que ele foi feito rapidamente, sem preocupação com qualidade de software ou manutenibilidade.
- Ao descobrir que o software precisará ser reescrito, o cliente exige que o protótipo vire o produto final, resultando em um sistema frágil.

2. Más escolhas do desenvolvedor

- Para fazer o protótipo funcionar rápido, o desenvolvedor faz concessões (algoritmos ineficientes, linguagem inadequada, etc.).
- Se o protótipo evoluir, essas más escolhas temporárias podem se consolidar, tornando-se parte permanente da arquitetura do sistema.

Solução: As regras do jogo devem ser definidas **antes**. O cliente deve concordar que o protótipo servirá apenas para definir requisitos e será (total ou parcialmente) descartado depois.

Como todo sistema complexo, **o software evolui ao longo do tempo.**

Durante o avanço do projeto, os requisitos do negócio e do produto frequentemente mudam, tornando um planejamento *em linha reta* (linear) inadequado.

Cenário típico:

- **Prazos apertados** tornam impossível lançar um sistema completo de imediato.
- Pode ser viável lançar uma **versão limitada** inicial para atender à pressão comercial ou da concorrência.
- Versões refinadas são lançadas posteriormente após um melhor entendimento do sistema.

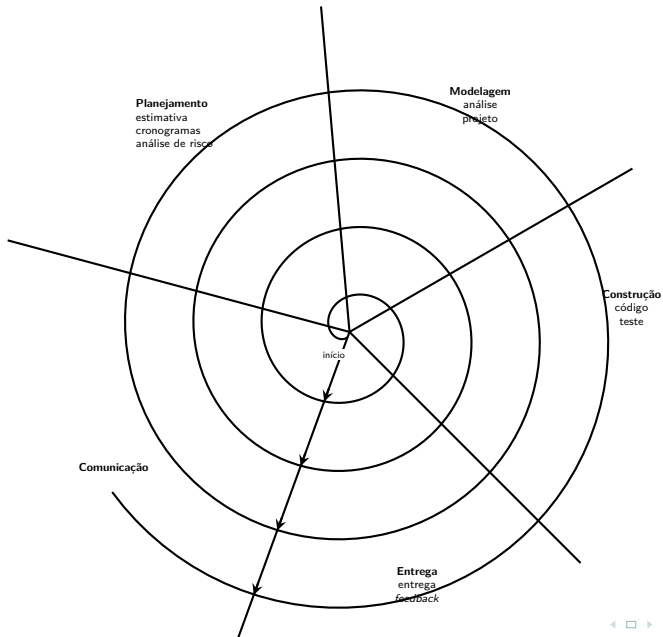
Conclusão: Nesses casos, faz-se necessário um modelo de processo projetado especificamente para desenvolver um produto que **crece e muda**.

O Modelo Espiral (Barry Boehm)

O **modelo espiral** é um processo de software evolucionário que une a natureza iterativa da **prototipação** aos aspectos sistemáticos e controlados do **modelo cascata**.

- **Versões Evolucionárias:** Nas primeiras iterações (voltas da espiral), a versão pode ser apenas um modelo ou protótipo. Em iterações posteriores, produz-se versões cada vez mais completas.
- **Dinâmica do Circuito:** A equipe realiza as atividades em um circuito **no sentido horário**, começando pelo centro.
- **Análise de Riscos:** Os riscos técnicos e de negócio são levados em conta à medida que cada revolução (volta) é realizada.
- **Marcos de Pontos-Âncora:** Cada passagem evolucionária atinge pontos-âncora (condições satisfeitas e artefatos gerados) ao longo da espiral.

Modelo de Processo Espiral



- **Evolução Progressiva:**

- O *primeiro circuito* (mais interno) gera uma **especificação** de produto.
- *Passagens subsequentes* desenvolvem protótipos e, em seguida, versões cada vez mais sofisticadas do software.

- **Ajustes Contínuos:** Cada passagem pela região de planejamento resulta em reajustes. Custo, cronograma e o número de iterações são revistos com base no **feedback do cliente**.

- **Ciclo de Vida Extendido:** Diferente de outros modelos que terminam na entrega, o modelo espiral pode ser aplicado **ao longo de toda a vida** do software.

Principais Vantagens

- Abordagem realista para sistemas e software em **larga escala**.
- Exige consideração direta e reduz os **riscos técnicos** antes que se tornem problemáticos, utilizando a prototipação.

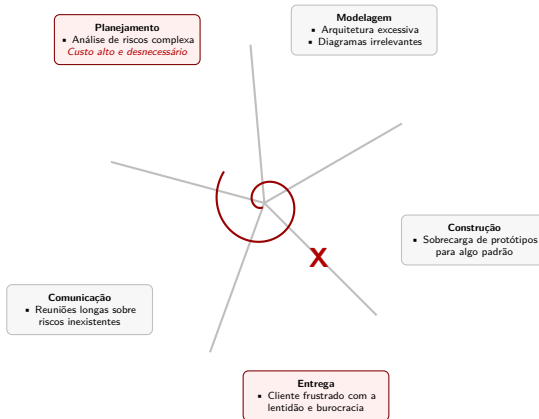
Desafios e Problemas

- Pode ser difícil convencer clientes (em situações contratuais) de que a abordagem evolucionária é **controlável**.
- Exige considerável **especialização na avaliação de riscos**. Se um grande risco passar despercebido, ocorrerão problemas sérios.

Contexto Atual: O tempo de colocação no mercado (*time-to-market*) muitas vezes é o requisito mais crítico. Se o momento oportuno for perdido, o projeto de software pode perder o sentido. O modelo espiral ajuda a entregar valor de forma cadenciada.

Exemplo: Espiral **Inadequado**

Cenário: Criação de um *Website Institucional Simples* (Baixa complexidade, requisitos padronizados).



Por que falha? Projetos **simples** e de **baixo risco** não necessitam da sobrecarga de análise. O método espiral para estes casos gera burocracia, estoura o orçamento e atrasa a entrega.